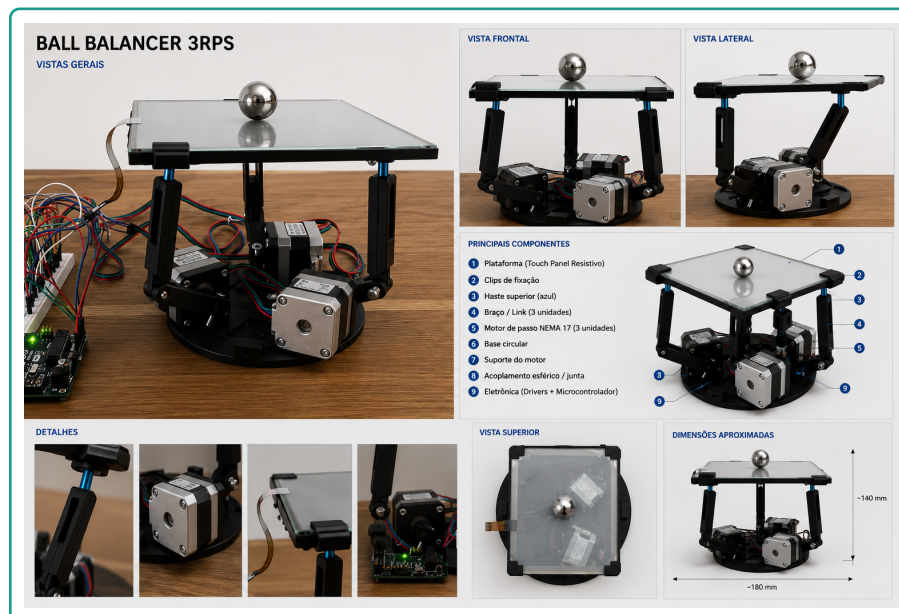
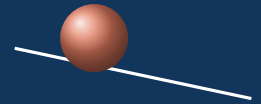


APOSTILA DE CONTROLE

Do duplo integrador ao controle digital

aplicado ao Ball Balancer 3RPS (ESP32-S3)



Uma apostila que parte das **leis de Newton** e chega ao **PID discreto embarcado**, sempre conectando cada equação à mesa real: a planta $P(s) = A/s^2$, os ganhos K_p, K_i, K_d , a cinemática inversa 3RPS e o firmware em C rodando a 100 Hz no ESP32-S3.

Como usar esta apostila

Esta apostila foi escrita para quem está construindo (e quer *dominar*) o **Ball Balancer 3RPS** — uma mesa que equilibra uma bola no centro, inclinando-se por três motores de passo sob comando de um controlador PID digital embarcado. Cada capítulo abre com a teoria geral e termina amarrando-a ao **seu** sistema: os números reais, o código real e as decisões de projeto reais.

Três tipos de caixa para guiar a leitura

Caixas azuis (Conceito) fixam a definição formal; **caixas verdes** (No projeto) mostram onde aquilo aparece no Ball Balancer; **caixas laranjas** (Exemplo numérico) fazem a conta com os parâmetros reais; **caixas roxas** (Intuição) traduzem a matemática em imagem mental; **caixas vermelhas** (Atenção) alertam para armadilhas.

Pré-requisitos: cálculo (derivada e integral), noções de equações diferenciais e álgebra linear básica. Não é preciso ter visto Controle antes — construiremos tudo do zero.

Sumário

Como usar esta apostila	i
Lista de Símbolos	vii
I Fundamentos Físicos e Modelagem	1
1 Introdução ao Problema	2
1.1 O que é um Ball Balancer	2
1.2 Por que é um problema clássico de controle	2
1.3 Por que é naturalmente instável	2
1.4 Os desafios do controle	3
2 Revisão Matemática Compacta	4
2.1 Derivada — taxa de variação	4
2.2 Integral — acumulação	4
2.3 Equações diferenciais	4
2.4 Variáveis de estado	4
2.5 Sistema dinâmico	5
3 Modelagem Física da Esfera	6
3.1 Cenário e forças atuantes	6
3.2 Segunda Lei de Newton (translação)	6
3.3 Equação de Euler (rotação)	6
3.4 Rolamento sem escorregamento	6
3.5 Eliminando o atrito: a origem do 5/7	7
3.6 Comparando esfera, cilindro e anel	7
4 Linearização	9
4.1 A aproximação seno de alfa aproximadamente alfa	9
4.2 Quando a aproximação é válida	9
4.3 O que acontece se ela não valer	9
5 Transformada de Laplace	11
5.1 O que é e por que usamos	11
5.2 Derivando $P(s)=A/s^2$ passo a passo	11
6 Função de Transferência da Planta	12
6.1 O que é uma “planta”	12
6.2 Polos e zeros: significado físico	12
6.3 O que significam dois polos na origem	13
II Análise de Sistemas de Controle	14
7 Diagramas de Blocos	15
7.1 Sistema real (cascata completa)	15
7.2 Sistema simplificado (controle puro)	15

7.3	Malha fechada e realimentação negativa	15
8	Estabilidade	16
8.1	Estável, instável, marginalmente estável	16
8.2	Onde a planta se encaixa	16
8.3	Como o controlador estabiliza	16
9	O Controlador PID	17
9.1	A lei de controle	17
9.2	K_p — proporcional	17
9.3	K_d — derivativo	18
9.4	K_i — integral	18
10	Como os Ganhos Foram Obtidos	19
10.1	Três caminhos para escolher K_p , K_i , K_d	19
10.2	Alocação de polos passo a passo	19
10.2.1	Dos requisitos a zeta e omega n	19
10.3	Comparativo dos conjuntos de ganhos do projeto	19
11	Zeta e omega n	22
11.1	Frequência natural omega n	22
11.2	Coefficiente de amortecimento zeta	22
12	Sobressinal (Overshoot)	24
12.1	O que é	24
12.2	Relação com zeta e fórmula	24
13	Tempos de Resposta	25
14	Método de Ziegler–Nichols	26
14.1	A ideia	26
III	Controle Digital	28
15	Fundamentos de Controle Digital	29
15.1	Amostragem e período T	29
15.2	Quantização	29
15.3	Teorema de Nyquist e <i>aliasing</i>	29
16	Atrasos no Sistema	30
16.1	As três fontes de atraso	30
16.2	Por que atraso reduz a estabilidade	30
17	O Domínio Z	31
17.1	O que é e por que existe	31
17.2	Relação entre s e z	31
18	Discretização	32
19	O PID Discreto	33
19.1	Do contínuo ao discreto, termo a termo	33
19.2	A equação implementada	33

20 Filtro Derivativo	34
20.1 O problema: o derivativo amplifica ruído	34
20.2 A solução: passa-baixa de primeira ordem	34
21 Anti-Windup	35
21.1 Saturação e <i>windup</i>	35
21.2 Back-calculation	35
IV Implementação no Projeto	36
22 O Sensor: a Tela Resistiva	37
22.1 Como funciona	37
22.2 Conversão para coordenadas	37
22.3 Calibração	37
22.4 Ruído e quantização	37
22.5 Impacto no controle	38
23 Análise do Código	39
23.1 <code>pid.c</code> — o controlador por eixo	39
23.2 <code>control.c</code> — o laço de 100 Hz	39
23.3 <code>kinematics.c</code> — a cinemática inversa 3RPS	39
24 Arquitetura Completa do Sistema	41
24.1 O fluxo de ponta a ponta	41
24.2 Camadas de software	41
24.3 O atuador: perfil trapezoidal	41
25 Guia Prático de Sintonia	42
25.1 Roteiro recomendado	42
25.2 Reconhecendo cada patologia	42
25.2.1 Assinaturas visuais (o que olhar na bancada)	42
25.3 Teoria vs. Hardware: por que os ganhos não batem	43
26 Melhorias Futuras	44
26.1 Feedforward (já presente: o TRIM)	44
26.2 Para além do PID (menção breve)	44
Referências	46

Lista de Figuras

1.1	Princípio do Ball Balancer: inclinar o tampo gera a força que move a bola. . . .	2
3.1	Diagrama de corpo livre da esfera rolando no plano inclinado.	6
6.1	Plano- s da planta: dois polos exatamente na origem (fronteira da estabilidade).	12
7.1	Cadeia física completa: sensor $\rightarrow$ PID $\rightarrow$ cinemática $\rightarrow$ motores $\rightarrow$ plataforma $\rightarrow$ bola.	15
7.2	Malha de controle reduzida ao essencial: controlador $C(s)$ sobre a planta $P(s)$	15
9.1	Mais derivativo $\Rightarrow$ mais amortecimento: a oscilação some à custa de um pouco de velocidade.	18
11.1	Quanto menor ζ , maior o sobressinal e mais oscilações; $\zeta = 1$ é o limiar sem overshoot.	22
13.1	Resposta ao degrau com t_p e a faixa de acomodação de $\pm 2\%$	25
14.1	No limite, o erro oscila com amplitude constante e período T_u	26
17.1	O mapa $z = e^{sT}$ “enrola” o semiplano estável de s no interior do círculo unitário de z	31
21.1	O anti-windup impede o integrador de “inflar” durante a saturação (ilustrativo).	35
24.1	Arquitetura completa: o anel fecha do sensor à bola e volta, a 100 Hz.	41

Lista de Tabelas

3.1	Quanto da gravidade vira “andar” para cada corpo que rola.	7
4.1	Qualidade da aproximação $\sin \alpha \approx \alpha$	9
6.1	Onde está o polo $\Rightarrow$ como o sistema reage (com o exemplo da bola).	13
9.1	Efeito de aumentar cada ganho sobre a trajetória da bola.	17
10.1	Conjuntos de ganhos: projeto (analítico), default do firmware e o que está em operação.	20
12.1	Overshoot em função de ζ	24
14.1	Tabela clássica de Ziegler–Nichols (malha fechada).	26
16.1	Ordem de grandeza dos atrasos no Ball Balancer.	30
18.1	Métodos de discretização (aproximações de s).	32
24.1	Módulos do firmware e seu papel.	41
25.1	Diagnóstico rápido por sintoma.	42
25.2	Por que o ganho de bancada difere do ganho de projeto.	43
26.1	Quando valeria a pena cada técnica avançada.	44

Lista de Símbolos

Símbolo	Significado	Valor típico
x, y	Posição da bola na mesa (eixos X e Y)	mm
r	Referência / setpoint (centro da mesa)	(93,5; 70,5) mm
e	Erro de posição, $e = x - r$	mm
u, α	Saída do controlador = inclinação da mesa	rad
g	Aceleração da gravidade	9810 mm/s ²
A	Ganho da planta, $A = \frac{5}{7}g$	≈ 7007 mm/s ²
$P(s)$	Função de transferência da planta	A/s^2
$C(s)$	Função de transferência do controlador (PID)	—
K_p, K_i, K_d	Ganhos proporcional, integral e derivativo	ver Cap. 10
ζ	Coefficiente de amortecimento	0– ∞
ω_n	Frequência natural não amortecida	rad/s
ω_d	Frequência natural amortecida, $\omega_n \sqrt{1 - \zeta^2}$	rad/s
M_p	Sobressinal (overshoot) máximo	%
t_s, t_r, t_p	Tempos de acomodação, subida e de pico	s
T	Período de amostragem	0,01 s (100 Hz)
τ_d	Constante do filtro derivativo	0,03 s
z	Variável da Transformada Z	—
$\theta_A, \theta_B, \theta_C$	Ângulos dos três motores (cinemática 3RPS)	graus
(n_x, n_y)	Componentes do vetor normal da mesa	—

Parte I

Fundamentos Físicos e Modelagem

CAPÍTULO 1

Introdução ao Problema

1.1 O que é um Ball Balancer

Um **Ball Balancer** (mesa equilibrista) é um sistema cujo único objetivo é manter uma **bola** parada num ponto de uma **superfície** que pode ser **inclinada**. Quando a mesa se inclina, a componente da gravidade ao longo do tampo acelera a bola; o controlador deve inclinar a mesa *na medida certa* para frear a bola e trazê-la de volta ao centro.

No projeto Ball Balancer

No nosso rig, a superfície é uma **tela resistiva de 4 fios** (187×141 mm) que mede a posição da bola; três **motores de passo NEMA 17** em arranjo **3RPS** inclinam a plataforma; e um **ESP32-S3** roda dois controladores PID (um para X, outro para Y) a 100 Hz.

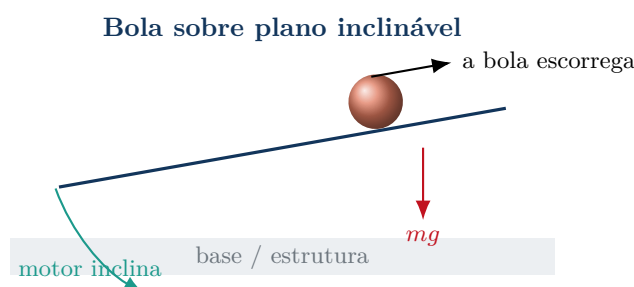


Figura 1.1: Princípio do Ball Balancer: inclinar o tampo gera a força que move a bola.

1.2 Por que é um problema clássico de controle

O Ball Balancer (e seus primos *ball-and-beam*, *ball-on-plate*) é um laboratório didático perfeito porque reúne, num único sistema barato e visível a olho nu:

- uma planta **naturalmente instável** (a bola foge sozinha);
- dinâmica de **segunda ordem “pura”** (duplo integrador), ideal para enxergar ζ e ω_n ;
- necessidade clara dos três termos **P, I e D**;
- realimentação por **sensor real** (ruído, atraso, quantização);
- **atuadores reais** (motores com limite de velocidade e saturação).

1.3 Por que é naturalmente instável

Deixe a mesa parada e levemente torta: a bola **acelera** e **nunca para por conta própria** — ela rola até cair. Não existe “força de mola” que a traga de volta. Em linguagem de controle, a planta tem **dois polos na origem** (Cap. 6): qualquer perturbação cresce sem limite. É o

controlador que *cria*, artificialmente, a rigidez (via K_p) e o amortecimento (via K_d) que a física não oferece.

1.4 Os desafios do controle

1. **Estabilizar** algo que sozinho diverge (Cap. 8).
2. **Amortecer** sem deixar lento demais — equilíbrio entre K_p e K_d .
3. **Eliminar o desvio** causado pela mesa torta — papel do K_i (e do *feedforward* TRIM, Cap. 26).
4. **Rejeitar ruído** do sensor sem amplificá-lo no derivativo (Cap. 20).
5. **Respeitar limites** físicos: saturação de inclinação ($\pm 0,25$ rad) e de velocidade dos motores (anti-windup, Cap. 21).
6. Fazer tudo isso em **tempo discreto**, a cada 10 ms (Parte III).

CAPÍTULO 2

Revisão Matemática Compacta

Antes de modelar a bola, vale alinhar cinco ferramentas. A meta aqui não é rigor, e sim **intuição**: o que cada conceito *significa* para o nosso sistema.

2.1 Derivada — taxa de variação

A derivada mede **quão rápido** uma grandeza muda. Se $x(t)$ é a posição da bola,

$$\dot{x} = \frac{dx}{dt} \text{ (velocidade)}, \quad \ddot{x} = \frac{d^2x}{dt^2} \text{ (aceleração)}.$$

Intuição

Derivar é perguntar “*está aumentando ou diminuindo, e com que pressa?*”. A velocidade é a derivada da posição; a aceleração, a derivada da velocidade. O termo **D** do PID usa exatamente isto: olha a *velocidade* da bola para antecipar para onde ela vai.

2.2 Integral — acumulação

A integral é o **oposto** da derivada: **acumula** (soma) a grandeza ao longo do tempo — a “área sob a curva”.

$$x(t) = x_0 + \int_0^t \dot{x}(\tau) d\tau.$$

Intuição

Sabendo a velocidade a cada instante e *somando*, descobre-se o quanto a bola andou. O termo **I** do PID acumula o *erro* no tempo — é a “memória” que percebe um desvio teimoso (mesa torta) e o corrige.

2.3 Equações diferenciais

Uma **equação diferencial** relaciona uma função com suas derivadas. A física da bola é uma delas:

$$\ddot{x} = A\alpha \quad (\text{“a aceleração é proporcional à inclinação”}).$$

Resolvê-la é achar $x(t)$ a partir dessa regra — tarefa que a Transformada de Laplace (Cap. 5) cumpre de forma *algébrica*.

2.4 Variáveis de estado

O **estado** é o conjunto mínimo de números que resume tudo o que o sistema precisa de si para evoluir. Para a bola num eixo,

$$\mathbf{x} = \begin{bmatrix} \text{posição} \\ \text{velocidade} \end{bmatrix}.$$

Intuição

Sabendo *onde* a bola está, *com que velocidade*, e a inclinação aplicada, o futuro fica determinado. “Os dois números que importam” são posição e velocidade — por isso a planta é de **segunda ordem**.

2.5 Sistema dinâmico

Um **sistema dinâmico** é algo cujo estado **evolui no tempo** segundo uma regra, em resposta a uma **entrada**:

entrada (inclinação α) $\rightarrow$ sistema (bola) $\rightarrow$ saída (posição x).

No projeto Ball Balancer

Todo este capítulo serve a uma pergunta: “*dada a inclinação que comando, para onde vai a bola — e como escolher a inclinação para que ela fique no centro?*”. Derivada, integral e equação diferencial são a linguagem; o PID é a resposta.

CAPÍTULO 3

Modelagem Física da Esfera

Vamos deduzir, do zero, a equação que governa a bola. O objetivo é chegar a

$$\ddot{x} = \frac{5}{7} g \sin \alpha \quad (3.1)$$

e entender *de onde vem* o fator $5/7$.

3.1 Cenário e forças atuantes

Considere uma esfera maciça de massa m e raio ρ que **rola sem escorregar** sobre um plano inclinado de ângulo α . Ao longo do tempo agem:

- a componente do peso $mg \sin \alpha$ (puxa a bola ladeira abaixo);
- a força de atrito estático F_{at} (no ponto de contato, é ela que faz a bola *girar* em vez de deslizar).

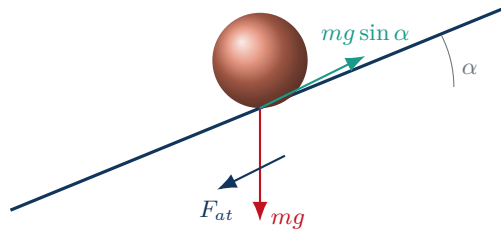


Figura 3.1: Diagrama de corpo livre da esfera rolando no plano inclinado.

3.2 Segunda Lei de Newton (translação)

Somando as forças ao longo do tempo (sentido positivo ladeira abaixo):

$$m \ddot{x} = mg \sin \alpha - F_{at}. \quad (3.2)$$

3.3 Equação de Euler (rotação)

A única força com braço de alavanca em torno do centro da bola é o atrito (braço = ρ). Pela 2.^a lei de Newton para a rotação,

$$I \dot{\omega} = F_{at} \rho, \quad (3.3)$$

onde I é o momento de inércia e ω a velocidade angular.

3.4 Rolamento sem escorregamento

A condição de rolar sem deslizar amarra translação e rotação:

$$\dot{x} = \omega \rho \implies \ddot{x} = \dot{\omega} \rho \implies \dot{\omega} = \frac{\ddot{x}}{\rho}. \quad (3.4)$$

3.5 Eliminando o atrito: a origem do 5/7

Para a **esfera maciça**, o momento de inércia em torno do centro é

$$I = \frac{2}{5} m \rho^2. \quad (3.5)$$

Substituindo (3.4) em (3.3):

$$F_{at} = \frac{I \dot{\omega}}{\rho} = \frac{1}{\rho} \left(\frac{2}{5} m \rho^2 \right) \frac{\ddot{x}}{\rho} = \frac{2}{5} m \ddot{x}. \quad (3.6)$$

Levando esse F_{at} de volta à equação de Newton (3.2):

$$m \ddot{x} = mg \sin \alpha - \frac{2}{5} m \ddot{x} \implies \left(1 + \frac{2}{5} \right) \ddot{x} = g \sin \alpha \implies \frac{7}{5} \ddot{x} = g \sin \alpha, \quad (3.7)$$

de onde sai, finalmente, a Eq. (3.1): $\ddot{x} = \frac{5}{7} g \sin \alpha$.

Por que apenas 5/7 da gravidade?

Parte da energia que a gravidade fornece não vira velocidade de *translação*: ela vai **girar** a bola. Uma esfera maciça “gasta” 2/7 do empurrão para rodar e sobra 5/7 para andar. Por isso uma bola desce a rampa mais devagar do que um bloco que desliza sem girar (esse teria fator 1). Um anel/casca esférica descera ainda mais devagar.

3.6 Comparando esfera, cilindro e anel

O fator depende só de *como a massa se distribui* (do momento de inércia $I = c m \rho^2$). Refazendo a dedução com um I genérico, a aceleração fica $\ddot{x} = \frac{g \sin \alpha}{1 + c}$:

Tabela 3.1: Quanto da gravidade vira “andar” para cada corpo que rola.

Corpo	I	fator $\frac{1}{1+c}$	desce...
Bloco deslizando (não gira)	0	1	mais rápido
Esfera maciça (nossa bola)	$\frac{2}{5} m \rho^2$	5/7 $\approx$ 0,71	—
Cilindro maciço	$\frac{1}{2} m \rho^2$	$2/3 \approx 0,67$	um pouco mais devagar
Anel / casca fina	$m \rho^2$	$1/2 = 0,50$	bem mais devagar

Para onde vai a energia da gravidade

A energia potencial perdida pela bola ao descer vira energia cinética, que se **divide** em duas partes:

$$\underbrace{mgh}_{\text{potencial}} = \underbrace{\frac{1}{2}mv^2}_{\text{translação (andar)}} + \underbrace{\frac{1}{2}I\omega^2}_{\text{rotação (girar)}}.$$

Quanto mais massa longe do centro (maior I), mais energia “escapa” para girar e menos sobra para andar. A esfera maciça concentra massa perto do centro, então guarda boa parte (5/7) para a translação — por isso é o corpo ideal para um Ball Balancer ágil.

O fator vive no código

Na simulação física, esse mesmo coeficiente aparece como `ROLL_FACTOR = 5.0/7.0` no arquivo `tools/PIDSimba.py`. A física do firmware e a da simulação usam exatamente a Eq. (3.1).

CAPÍTULO 4

Linearização

A Eq. (3.1) é **não linear** por causa do $\sin \alpha$. Ferramentas como Laplace, polos e ζ/ω_n exigem um modelo **linear**. A saída é a aproximação de pequenos ângulos.

4.1 A aproximação $\sin \alpha \approx \alpha$

A série de Taylor do seno em torno de zero é

$$\sin \alpha = \alpha - \frac{\alpha^3}{6} + \frac{\alpha^5}{120} - \dots \quad (4.1)$$

Para α pequeno (em **radianos**), o termo dominante é o próprio α , e os demais são desprezíveis. Logo $\sin \alpha \approx \alpha$ e a planta vira **linear**:

$$\ddot{x} \approx \frac{5}{7} g \alpha. \quad (4.2)$$

4.2 Quando a aproximação é válida

Erro da aproximação no limite de operação

O firmware satura a inclinação em $|\alpha| \leq 0,25$ rad ($\approx 14,3^\circ$). No pior caso:

$$\sin(0,25) = 0,24740, \quad \text{erro relativo} = \frac{0,25 - 0,24740}{0,24740} \approx 1,05\%.$$

Ou seja, mesmo no **extremo** do curso a linearização erra cerca de 1% — e na operação normal (poucos graus) o erro é bem menor que isso.

Tabela 4.1: Qualidade da aproximação $\sin \alpha \approx \alpha$.

α (graus)	α (rad)	$\sin \alpha$	erro relativo
1°	0,0175	0,01745	0,005%
5°	0,0873	0,08716	0,13%
10°	0,1745	0,17365	0,51%
$14,3^\circ$	0,2497	0,24740	1,05%

4.3 O que acontece se ela não valer

Limitação consciente do projeto

Para ângulos grandes, $\sin \alpha < \alpha$: a mesa entregaria *menos* aceleração do que o modelo linear supõe, e o controlador projetado ficaria “otimista” (ganho efetivo menor que o previsto). Por isso a saturação em 0,25 rad não é só segurança mecânica: ela **mantém o sistema dentro da região onde a teoria linear vale**. Operar a mesa em ângulos

muito maiores invalidaria todo o projeto de ganhos da Parte II.

CAPÍTULO 5

Transformada de Laplace

5.1 O que é e por que usamos

A **Transformada de Laplace** converte uma função do tempo $f(t)$ numa função de uma variável complexa s :

$$F(s) = \mathcal{L}\{f(t)\} = \int_0^\infty f(t) e^{-st} dt. \quad (5.1)$$

A mágica: ela transforma **derivadas e integrais** (operações do cálculo) em **multiplicações e divisões** por s (operações de álgebra). Uma equação diferencial vira uma equação algébrica.

As duas propriedades que usaremos o tempo todo

Com condições iniciais nulas,

$$\mathcal{L}\{\dot{f}(t)\} = s F(s), \quad \mathcal{L}\left\{\int_0^t f d\tau\right\} = \frac{F(s)}{s}.$$

Derivar $\leftrightarrow$ multiplicar por s . **Integrar** $\leftrightarrow$ dividir por s .

5.2 Derivando $P(s) = A/s^2$ passo a passo

Partimos da planta linearizada (4.2), chamando a entrada de $u = \alpha$ (a inclinação comandada) e $A = \frac{5}{7}g$:

$$\ddot{x}(t) = A u(t). \quad (\text{equação no tempo}) \quad (5.2)$$

$$\mathcal{L}\{\ddot{x}(t)\} = \mathcal{L}\{A u(t)\}. \quad (\text{aplica Laplace}) \quad (5.3)$$

$$s^2 X(s) = A U(s). \quad (\text{derivada segunda} \rightarrow s^2) \quad (5.4)$$

$$\frac{X(s)}{U(s)} = \frac{A}{s^2}. \quad (\text{isola a razão saída/entrada}) \quad (5.5)$$

$$P(s) = \frac{X(s)}{U(s)} = \frac{A}{s^2}, \quad A \approx 7007 \text{ mm/s}^2 \quad (5.6)$$

De onde sai A aproximadamente 7007

$A = \frac{5}{7}g$ com $g = 9810 \text{ mm/s}^2$ (gravidade em milímetros, pois medimos a bola em mm). Logo $A = \frac{5}{7} \cdot 9810 \approx 7007 \text{ mm/s}^2$ por radiano de inclinação. Esse número é a “alavanca” que liga inclinação a aceleração em todo o projeto.

CAPÍTULO 6

Função de Transferência da Planta

6.1 O que é uma “planta”

Planta é o sistema físico que queremos controlar — aqui, “a bola respondendo à inclinação”. Sua função de transferência $P(s) = A/s^2$ resume *toda* a dinâmica: dada uma inclinação $U(s)$, ela diz qual será a posição $X(s)$.

Lendo $P(s)$

Numerador (A): é o “ganho/alavanca” da planta — *quanta* aceleração cada radiano de inclinação gera ($A \approx 7007 \text{ mm/s}^2$). Como não há s no numerador, **não há zeros**: a planta não “realça” nenhuma frequência, ela apenas integra.

Denominador (s^2): é a **dinâmica**. Cada s no denominador significa “integrar uma vez”; dois s significam integrar *duas* vezes.

Por que surgem *dois* integradores — e de forma natural

É pura física, não escolha de projeto. A inclinação gera **aceleração** ($\ddot{x} = A\alpha$); mas o que o sensor mede é **posição**. Para ir de uma à outra, a natureza *integra duas vezes*: aceleração $\rightarrow$ velocidade $\rightarrow$ posição. Esses dois “integrar” são, exatamente, os dois fatores $1/s$ — e os dois polos na origem.

6.2 Polos e zeros: significado físico

Definição

Polos são as raízes do *denominador*; ditam a **forma natural** da resposta (cresce? decai? oscila?). **Zeros** são as raízes do *numerador*; moldam *como* os modos são excitados. Em $P(s) = A/s^2$ o numerador é a constante A : **não há zeros**, e há um **polo duplo em $s = 0$** .

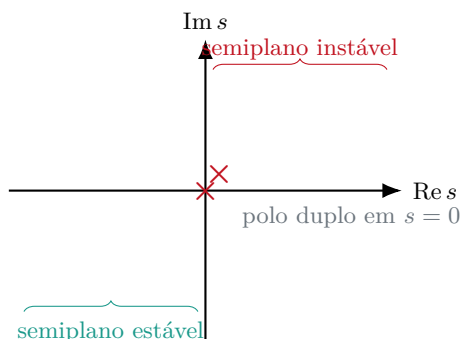


Figura 6.1: Plano- s da planta: dois polos exatamente na origem (fronteira da estabilidade).

Polo

Um polo descreve como o sistema se comporta *sozinho*, sem ninguém mexer — e a posição dele no plano- s já conta a história toda.

Tabela 6.1: Onde está o polo $\Rightarrow$ como o sistema reage (com o exemplo da bola).

Polo...	Comportamento natural	No Ball Balancer
à esquerda ($\sigma < 0$)	decai sozinho (estável)	o alvo: o controlador puxa os polos para cá
na origem ($\sigma = 0$)	nem cresce nem decai (marginal)	a planta crua — a bola não volta nem cai sozinha
à direita ($\sigma > 0$)	cresce sem limite (instável)	sinal/ganho errado: a mesa <i>empurra</i> a bola para fora

6.3 O que significam dois polos na origem

Cada $1/s$ é um **integrador**. Dois polos na origem = **duplo integrador**: a entrada é integrada *duas vezes* até virar posição.

$$u \xrightarrow{\int} \dot{x} \text{ (velocidade)} \xrightarrow{\int} x \text{ (posição)}. \quad (6.1)$$

Por que a bola acelera “para sempre”?

Incline a mesa por um ângulo fixo α_0 . A aceleração $\ddot{x} = A\alpha_0$ é constante e não nula. Integrando uma vez, a **velocidade cresce linearmente** ($\dot{x} = A\alpha_0 t$); integrando de novo, a **posição cresce com o quadrado do tempo** ($x = \frac{1}{2}A\alpha_0 t^2$). Nada nessa cadeia “puxa a bola de volta”: sem realimentação, ela diverge. Esse é o significado físico, palpável, dos dois polos na origem.

Sistema do Tipo 2

Ter dois integradores torna a planta do **Tipo 2**. Consequência positiva: em malha fechada, o erro estacionário a uma **referência constante** e até a uma **rampa** é *nulo* — antes mesmo de adicionar o termo integral do PID. O K_i , aqui, serve principalmente contra **perturbações** (a mesa torta), não contra erro de referência.

Parte II

Análise de Sistemas de Controle

CAPÍTULO 7

Diagramas de Blocos

Diagramas de blocos mostram o **fluxo de sinais**. Vamos do mais detalhado (o sistema físico real) ao mais abstrato (malha fechada clássica).

7.1 Sistema real (cascata completa)

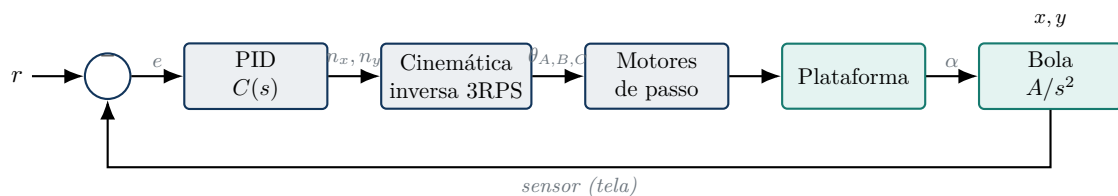


Figura 7.1: Cadeia física completa: sensor → PID → cinemática → motores → plataforma → bola.

7.2 Sistema simplificado (controle puro)

A cinemática e os motores são rápidos e essencialmente **estáticos** perante a dinâmica lenta da bola (Cap. 24): podemos colocá-los como ganho unitário e enxergar o miolo do controle.

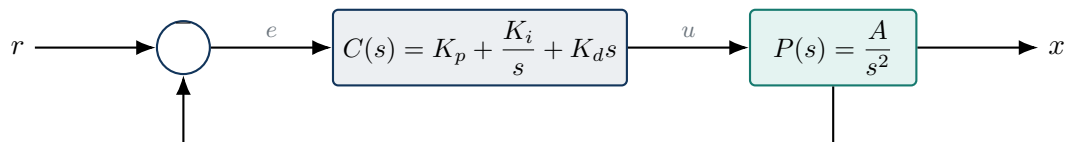


Figura 7.2: Malha de controle reduzida ao essencial: controlador $C(s)$ sobre a planta $P(s)$.

7.3 Malha fechada e realimentação negativa

A função de transferência de malha fechada (entrada r , saída x) é

$$\frac{X(s)}{R(s)} = \frac{C(s)P(s)}{1 + C(s)P(s)}. \quad (7.1)$$

O sinal de **menos** no somador (realimentação *negativa*) é o que faz o sistema *corrigir* o erro em vez de amplificá-lo.

O sinal no firmware: TILT_SIGN = -1

O PID calcula $u = K_p e + \dots$ com $e = \text{medida} - \text{setpoint}$. Para a mesa inclinar no sentido que traz a bola de volta, aplica-se $n_x = -u$ (TILT_SIGN_X = -1.0f em control.c). Se um eixo empurrar a bola para longe, basta inverter aquele sinal ao vivo: SX +1 ou SY +1. É a Eq. (7.1) ganhando vida no hardware.

CAPÍTULO 8

Estabilidade

8.1 Estável, instável, marginalmente estável

Critério pelos polos de malha fechada

Um sistema linear é **estável** se *todos* os polos de malha fechada têm parte real **negativa** (semiplano esquerdo); **instável** se algum tem parte real **positiva**; **marginalmente estável** se há polos *exatamente* sobre o eixo imaginário (e nenhum à direita).

8.2 Onde a planta se encaixa

A planta isolada A/s^2 tem polo duplo em $s = 0$ — **em cima** do eixo imaginário: *marginalmente estável* no melhor caso e, na prática (qualquer perturbação), divergente. **Sem controlador, a mesa não funciona.**

8.3 Como o controlador estabiliza

Feche a malha com um **PD** ($C(s) = K_p + K_d s$) sobre A/s^2 . A equação característica $1 + C(s)P(s) = 0$ vira

$$s^2 + AK_d s + AK_p = 0. \quad (8.1)$$

Pelo critério de Routh–Hurwitz, um polinômio de 2.^a ordem $s^2 + a_1 s + a_0$ tem ambas as raízes no semiplano esquerdo *se e somente se* $a_1 > 0$ e $a_0 > 0$. Aqui $a_1 = AK_d$ e $a_0 = AK_p$: basta $K_p > 0$ e $K_d > 0$.

O ganho derivativo traz a estabilidade

Sem o termo derivativo ($K_d = 0$), a Eq. (8.1) vira $s^2 + AK_p = 0$, com raízes $\pm j\sqrt{AK_p}$ — *de novo* sobre o eixo imaginário: oscilação não amortecida. **É o K_d que empurra os polos para a esquerda** e amortece. Guarde isto: num duplo integrador, derivativo não é “tempero”, é requisito de estabilidade.

CAPÍTULO 9

O Controlador PID

9.1 A lei de controle

$$u(t) = \underbrace{K_p e(t)}_{\text{presente}} + \underbrace{K_i \int_0^t e(\tau) d\tau}_{\text{passado}} + \underbrace{K_d \dot{e}(t)}_{\text{futuro}}, \quad C(s) = K_p + \frac{K_i}{s} + K_d s. \quad (9.1)$$

Cada termo olha para um “tempo”: o **P** reage ao erro *agora*, o **I** acumula o *passado*, e o **D** antecipa a tendência (*futuro*).

As três analogias mecânicas

A forma mais fácil de “sentir” o PID é imaginá-lo como três peças mecânicas agindo sobre a bola:

- K_p = **mola virtual**. Puxa a bola de volta com força proporcional ao *quanto* ela se afastou. Só mola, porém, oscila.
- K_d = **amortecedor**. Opõe-se à *velocidade* da bola, como um amortecedor de carro. É ele que “segura” a oscilação da mola.
- K_i = **memória acumulativa**. Lembra de um desvio que *insiste* (mesa torta) e vai inclinando mais até zerá-lo.

Mola + amortecedor + memória: é literalmente um sistema massa-mola-amortecedor que *você* projeta via K_p, K_d e ajusta o ponto de equilíbrio via K_i .

Tabela 9.1: Efeito de aumentar cada ganho sobre a trajetória da bola.

Ganho	Analogia	Se aumenta	Se falta
K_p	mola	mais rápido; em excesso, oscila	mole, lento, não chega
K_d	amortecedor	menos overshoot, mais firme	oscila /instabiliza; amplifica ruído se em excesso
K_i	memória	zera o desvio fixo	desvio fixo <i>persiste</i> ; em excesso, <i>windup</i>

9.2 K_p — proporcional

Interpretação física: rigidez de uma “mola virtual”. Quanto mais longe do centro, mais a mesa inclina para empurrar a bola de volta.

- **Aumenta** K_p : resposta mais rápida ($\omega_n = \sqrt{AK_p}$ sobe); em excesso, oscila e fica à beira da instabilidade.
- **Diminui** K_p : resposta mole e lenta; a bola demora a voltar.

9.3 K_d — derivativo

Interpretação física: amortecimento que reage à *velocidade* da bola. Se a bola corre para o centro rápido demais, o D “segura” antes que ela passe do ponto.

- **Aumenta K_d :** mais amortecimento, menos overshoot; em excesso, lentidão e **amplificação de ruído** (Cap. 20).
- **Diminui K_d :** mais overshoot e oscilação; em $K_d \rightarrow 0$, instabilidade marginal (Cap. 8).

9.4 K_i — integral

Interpretação física: memória que elimina desvio *persistente*. Se a bola insiste em ficar deslocada (mesa torta), o integral acumula esse erro e adiciona a inclinação constante que falta.

- **Aumenta K_i :** zera o desvio mais rápido; em excesso, causa *windup* e oscilação lenta.
- **Diminui K_i :** desvio some devagar (ou “nunca”, na prática).

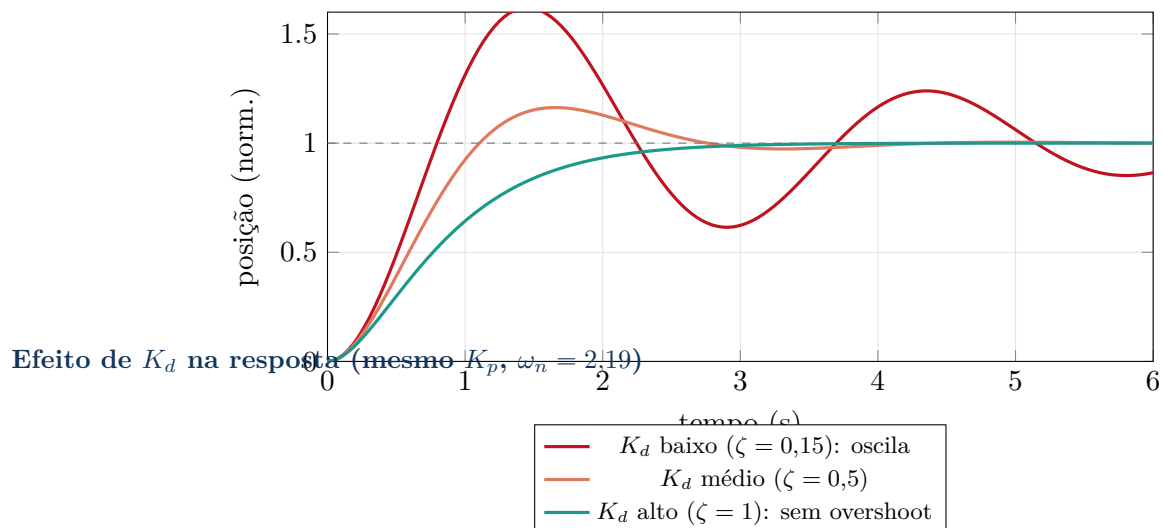


Figura 9.1: Mais derivativo $\Rightarrow$ mais amortecimento: a oscilação some à custa de um pouco de velocidade.

Os três termos no seu firmware

Em `pid.c`, a linha `out = p->kp*err + p->ki*p->integ + p->kd*deriv;` é literalmente $u = K_p e + K_i \int e + K_d \dot{e}$. O K_i combate a **base torta** (Cap. 26), e o firmware ainda oferece o **TRIM**, um *feedforward* que faz o mesmo trabalho do integral, porém instantaneamente.

CAPÍTULO 10

Como os Ganhos Foram Obtidos

10.1 Três caminhos para escolher K_p, K_i, K_d

1. **Alocação de polos** (analítico): escolhe-se *onde* os polos de malha fechada devem ficar (a partir de requisitos de ζ, ω_n) e resolve-se para os ganhos. É o método do TCC de referência [1].
2. **Ziegler–Nichols** (experimental): leva o sistema ao limite da oscilação e usa tabelas (Cap. 14).
3. **Tentativa e erro guiada** (prático): sobe K_p , depois K_d , depois um toque de K_i — é como se faz o ajuste fino no rig (Cap. 25).

10.2 Alocação de polos passo a passo

A malha fechada do PID sobre A/s^2 é de **terceira ordem**. Sua equação característica ($1 + C(s)P(s) = 0$, com $C = K_p + K_i/s + K_d s$) é

$$s^3 + AK_d s^2 + AK_p s + AK_i = 0. \quad (10.1)$$

Escolhemos um polinômio **desejado**: um par dominante de 2.^a ordem (define ζ, ω_n) mais um polo real p_o rápido:

$$(s^2 + 2\zeta\omega_n s + \omega_n^2)(s + p_o) = s^3 + (2\zeta\omega_n + p_o)s^2 + (\omega_n^2 + 2\zeta\omega_n p_o)s + \omega_n^2 p_o. \quad (10.2)$$

Igualando coeficiente a coeficiente com (10.1):

$$K_d = \frac{2\zeta\omega_n + p_o}{A}, \quad K_p = \frac{\omega_n^2 + 2\zeta\omega_n p_o}{A}, \quad K_i = \frac{\omega_n^2 p_o}{A} \quad (10.3)$$

10.2.1 Dos requisitos a ζ e ω_n

Requisitos do projeto: sobressinal $PO \leq 7,5\%$ e acomodação $t_s \leq 2,5$ s. Pelas fórmulas de 2.^a ordem (Cap. 11–13):

$$\zeta = \sqrt{\frac{\ln^2(PO/100)}{\pi^2 + \ln^2(PO/100)}} \approx 0,636, \quad \omega_n \approx 2,19 \text{ rad/s}. \quad (10.4)$$

Ganhos resultantes com po igual a 1 e A igual a 7007

Substituindo $\zeta = 0,636$, $\omega_n = 2,19$, $p_o = 1$ em (10.3):

$$K_d = \frac{2(0,636)(2,19) + 1}{7007} \approx 5,4 \times 10^{-4}, \quad K_p \approx 1,08 \times 10^{-3}, \quad K_i \approx 6,83 \times 10^{-4}.$$

Esses são os ganhos “de projeto” usados na simulação (`sim_pid.py`/PIDSimba.py).

10.3 Comparativo dos conjuntos de ganhos do projeto

Tabela 10.1: Conjuntos de ganhos: projeto (analítico), default do firmware e o que está em operação.

Conjunto	K_p	K_i	K_d	Observação
Projeto (alocação)	$1,08 \times 10^{-3}$	$6,83 \times 10^{-4}$	$5,4 \times 10^{-4}$	overshoot $\sim 25\%$ (3. ^a ordem)
“Melhorado” (sim)	$1,08 \times 10^{-3}$	$3,0 \times 10^{-4}$	$9,0 \times 10^{-4}$	menos K_i , mais K_d ($\zeta \approx 1,15$)
Default firmware	$8,0 \times 10^{-4}$	$2,0 \times 10^{-5}$	$1,2 \times 10^{-2}$	conservador, manso no boot
Em operação	$6,0 \times 10^{-4}$	$3,0 \times 10^{-5}$	$1,0 \times 10^{-4}$	afinado no rig (com TRIM)

Por que tantos conjuntos diferentes?

Os ganhos *analíticos* valem para o modelo ideal sem atraso. O hardware tem atraso de sensor, ruído e saturação, então os ganhos de *operação* são ajustados experimentalmente — e funcionam apoiados em dois aliados do firmware: a **rejeição de leituras espúrias** e o **TRIM**. Todos saem da mesma teoria; mudam as prioridades (ideal vs. robusto).

CAPÍTULO 11

ζ (Zeta) e ω_n

Todo sistema de 2.^a ordem padrão escreve-se como

$$\frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}. \quad (11.1)$$

Dois números resumem *tudo* sobre a resposta: ω_n e ζ .

11.1 Frequência natural ω_n

É a velocidade “de relógio” do sistema: quanto maior ω_n , mais rápida a resposta. Na nossa malha PD, $\omega_n = \sqrt{AK_p}$.

11.2 Coeficiente de amortecimento ζ

Mede *quanto* a oscilação é atenuada:

$\zeta = 0$	oscila para sempre (não amortecido)
$0 < \zeta < 1$	subamortecido (oscila e decai)
$\zeta = 1$	criticamente amortecido (mais rápido sem overshoot)
$\zeta > 1$	sobreamortecido (lento, sem overshoot)

Na malha PD, $\zeta = \frac{K_d}{2} \sqrt{\frac{A}{K_p}}$.

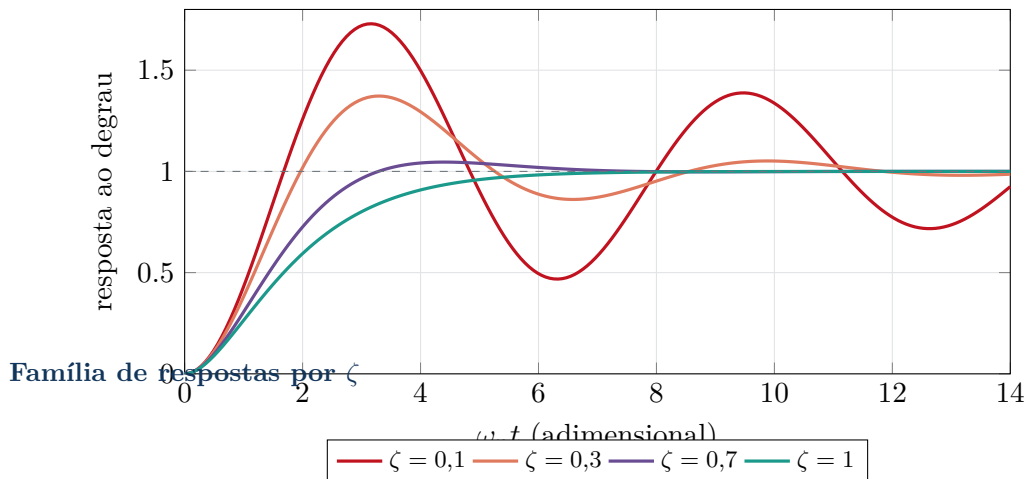


Figura 11.1: Quanto menor ζ , maior o sobressinal e mais oscilações; $\zeta = 1$ é o limiar sem overshoot.

Zeta dos conjuntos reais (parte PD)

Com $A = 7007$:

- **Default** ($K_p = 8 \times 10^{-4}$, $K_d = 1,2 \times 10^{-2}$): $\omega_n = \sqrt{7007 \cdot 8 \times 10^{-4}} = 2,37$ rad/s, $\zeta = \frac{1,2 \times 10^{-2}}{2} \sqrt{7007 / 8 \times 10^{-4}} \approx 17,8$ (muito lento).
- **Em operação** ($K_p = 6 \times 10^{-4}$, $K_d = 1 \times 10^{-4}$): $\omega_n = 2,05$ rad/s, $\zeta \approx 0,17$ (bem subamortecido).

Eis a leitura teórica do que você sente na mesa: com $\zeta \approx 0,17$, a resposta é rápida porém *oscilatória* — exatamente o regime onde a mesa “surta” sob qualquer perturbação se não houver as proteções do firmware.

O que voce ve na mesa quando zeta e omega n mudam

- ζ **baixo** (pouco K_d): a bola **passa do centro várias vezes**, vai-e-volta, demora a assentar — e “surta” a qualquer empurrão.
- $\zeta \approx 1$ (bom K_d): a bola chega ao centro **firme, sem passar do ponto**. É o alvo da sintonia.
- ζ **alto** (muito K_d): a bola volta **devagar, arrastada**, como se estivesse em mel.
- ω_n **maior** (mais K_p): **tudo fica mais rápido**, correções mais enérgicas. Mas como $\zeta = \frac{K_d}{2} \sqrt{A/K_p}$ *cai* quando K_p sobe, aumentar K_p *sozinho* deixa a bola rápida e mais oscilatória — por isso K_p e K_d são ajustados juntos.

CAPÍTULO 12

Sobressinal (Overshoot)

12.1 O que é

Overshoot (M_p) é o quanto a resposta *ultrapassa* o valor final antes de assentar. Na mesa: a bola passa do centro e volta.

12.2 Relação com ζ e fórmula

Para o sistema de 2.^a ordem subamortecido,

$$M_p = \exp\left(\frac{-\zeta\pi}{\sqrt{1-\zeta^2}}\right) \times 100\% \quad (12.1)$$

Invertendo (útil no projeto: do PO desejado para o ζ necessário):

$$\zeta = \frac{-\ln(M_p)}{\sqrt{\pi^2 + \ln^2(M_p)}}. \quad (12.2)$$

Verificando o requisito do projeto

Com $\zeta = 0,636$:

$$M_p = \exp\left(\frac{-0,636\pi}{\sqrt{1-0,636^2}}\right) = \exp\left(\frac{-1,998}{0,772}\right) = \exp(-2,589) \approx 0,075 = 7,5\%.$$

Confere com o requisito $PO \leq 7,5\%$ usado para escolher ζ no Cap. 10.

Tabela 12.1: Overshoot em função de ζ .

ζ	M_p
0,1	73%
0,3	37%
0,5	16%
0,636	7,5%
0,7	4,6%
1,0	0%

CAPÍTULO 13

Tempos de Resposta

Três tempos caracterizam “quão rápida” é a resposta.

Definições

t_r (**subida**): tempo para ir de $\sim 10\%$ a $\sim 90\%$ do valor final. t_p (**pico**): instante do sobressinal máximo. t_s (**acomodação**): tempo para entrar (e ficar) numa faixa de $\pm 2\%$ em torno do valor final.

$$t_p = \frac{\pi}{\omega_d} = \frac{\pi}{\omega_n \sqrt{1 - \zeta^2}}, \quad t_r \approx \frac{1,8}{\omega_n}, \quad t_s \approx \frac{4}{\zeta \omega_n} \text{ (critério de 2\%)}. \quad (13.1)$$

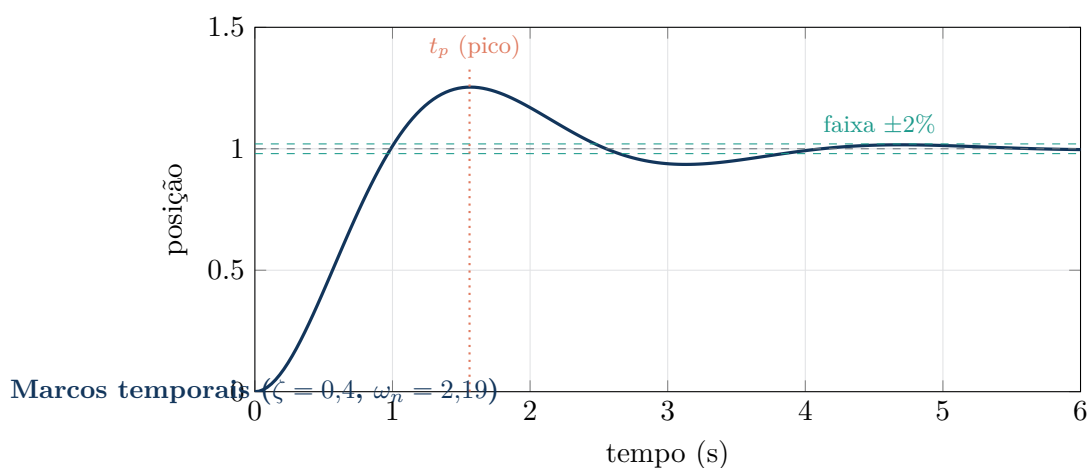


Figura 13.1: Resposta ao degrau com t_p e a faixa de acomodação de $\pm 2\%$.

Tempos do projeto

Com $\zeta = 0,636$ e $\omega_n = 2,19$: $\omega_d = 2,19\sqrt{1 - 0,636^2} = 1,69$ rad/s, $t_p = \pi/1,69 \approx 1,86$ s, $t_s \approx 4/(0,636 \cdot 2,19) \approx 2,87$ s. Coerente com o alvo $t_s \leq 2,5\text{--}3$ s.

CAPÍTULO 14

Método de Ziegler–Nichols

14.1 A ideia

Quando não se tem o modelo, Ziegler–Nichols (ZN) fornece ganhos a partir de **um experimento**: leva-se o sistema ao limite da estabilidade.

Ku e Tu

Com apenas o termo P ($K_i = K_d = 0$), aumente K_p até a saída entrar em **oscilação sustentada** (amplitude constante). Esse ganho é o **ganho crítico** K_u ; o período dessa oscilação é o **período crítico** T_u .

Tabela 14.1: Tabela clássica de Ziegler–Nichols (malha fechada).

Controlador	K_p	T_i	T_d
P	$0,50 K_u$	—	—
PI	$0,45 K_u$	$0,83 T_u$	—
PID	$0,60 K_u$	$0,50 T_u$	$0,125 T_u$

(Lembrando: $K_i = K_p/T_i$ e $K_d = K_p T_d$.)

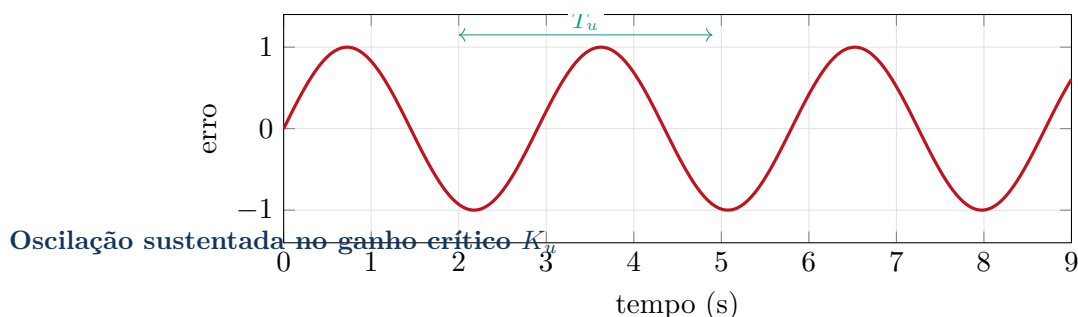


Figura 14.1: No limite, o erro oscila com amplitude constante e período T_u .

Como você faria o experimento de ZN na mesa

1. Pelo terminal: PID <kp> 0 0 (zera I e D). Ligue ERROR ou SHOW para acompanhar.
2. Suba K_p aos poucos (K_p 0.0005, K_p 0.0008, ...) até a bola **oscilar com amplitude constante** em torno do centro. Esse é K_u .
3. Meça o **período** dessa oscilação com cronômetro ou pelo *timestamp* da telemetria: é T_u .
4. Calcule $K_p = 0,6K_u$, $T_i = 0,5T_u$, $T_d = 0,125T_u$ e converta para K_i , K_d . Use como **chute inicial** e refine (Cap. 25).

Atencao

ZN tende a dar respostas **agressivas** (overshoot $\sim 25\%$). É excelente ponto de partida, mas quase sempre convém **aumentar** K_d (ou reduzir K_p) depois para amortecer — coerente com nossa planta sensível ao derivativo.

Parte III

Controle Digital

CAPÍTULO 15

Fundamentos de Controle Digital

O ESP32 não resolve integrais contínuas: ele executa o controlador em **instantes discretos**, a cada T segundos. Isso traz quatro conceitos.

15.1 Amostragem e período T

A cada T o firmware lê o sensor, calcula u e comanda os motores.

No projeto Ball Balancer

Nosso laço roda a **100 Hz** (`LOOP_HZ = 100`), logo $T = 0,01$ s. Subimos de 50 para 100 Hz justamente para **reduzir o atraso de amostragem pela metade** e melhorar a resposta.

15.2 Quantização

O ADC da tela tem resolução finita (12 bits, 0–4095); a posição em mm vem em “degraus”. Os motores também são discretos (passos). Isso aparece como pequenos “micro-movimentos” perto do centro — limite físico, não bug.

15.3 Teorema de Nyquist e *aliasing*

Nyquist

Para reconstruir um sinal sem ambiguidade, a frequência de amostragem deve ser **maior que o dobro** da maior frequência presente: $f_s > 2f_{max}$. A metade de f_s é a **frequência de Nyquist**.

Aliasing é o que ocorre se essa regra é violada: componentes rápidas “se disfarçam” de lentas e contaminam a medida.

Folga de amostragem no nosso caso

$f_s = 100$ Hz $\Rightarrow$ Nyquist = 50 Hz. A dinâmica da bola tem $\omega_n \approx 2,19$ rad/s, ou seja $f_n = \omega_n/2\pi \approx 0,35$ Hz. A amostragem é mais de **140 vezes** mais rápida que a dinâmica — folga enorme, e a aproximação “contínua” dos capítulos anteriores vale com segurança.

CAPÍTULO 16

Atrasos no Sistema

Todo sistema real **demora** a reagir. Entre a bola se mover e a mesa responder de fato, passam-se alguns milissegundos — e atraso, em controle, é **inimigo da estabilidade**. Vejamos de onde vem e por quê.

16.1 As três fontes de atraso

1. **Atraso de amostragem.** O firmware só “olha” a bola a cada $T = 10$ ms. Em média, a leitura usada já tem $T/2 = 5$ ms de idade.
2. **Atraso de processamento.** Ler o ADC, filtrar, rodar os dois PIDs e a cinemática consome tempo de CPU; no nosso caso, pequeno perante os 10 ms do laço.
3. **Atraso do atuador.** O motor não salta para o ângulo: ele **acelera e desacelera** (perfil trapezoidal). Entre comandar a inclinação e ela *acontecer* passam-se alguns milissegundos.

Tabela 16.1: Ordem de grandeza dos atrasos no Ball Balancer.

Fonte	Ordem	Como o projeto mitiga
Amostragem	~ 5 ms ($T/2$)	subir a taxa: 50 $\rightarrow$ 100 Hz (metade do atraso)
Processamento	< 1 ms	código enxuto; telemetria limitada
Filtro (IIR/derivada)	~ 10 –25 ms	α ajustado (ruído $\times$ atraso)
Atuador (rampa)	poucos ms	aceleração alta o bastante, sem perder passo

16.2 Por que atraso reduz a estabilidade

Dirigir olhando pelo retrovisor

Imagine corrigir a direção de um carro vendo a estrada com 1 segundo de **atraso**. Você reage ao que *já passou*: corrige tarde, passa do ponto, corrige de novo — e começa a serpentear. O atraso transforma um controle calmo em oscilatório.

Tecnicamente, o atraso **consome margem de fase**: o termo derivativo, que deveria *antecipar* o movimento, passa a agir sobre informação *velha* e perde eficácia. Por isso não adianta só “encher de K_d ”: com muito atraso, nem o derivativo segura a oscilação.

Como o projeto combate o atraso

- **Laço a 100 Hz** (em vez de 50): corta o atraso de amostragem pela metade — melhoria direta de resposta.
- **Filtro derivativo com τ_d moderado**: corta ruído sem adicionar atraso demais (Cap. 20).
- **Rejeição de outliers** no lugar de filtragem pesada: remove o *glitch* **sem** a lentidão de um filtro forte.

CAPÍTULO 17

O Domínio Z

17.1 O que é e por que existe

Assim como Laplace (s) é a linguagem natural do tempo *contínuo*, a **Transformada Z** (z) é a do tempo *discreto*. Ela transforma sequências $f[k]$ em funções de z :

$$F(z) = \sum_{k=0}^{\infty} f[k] z^{-k}. \quad (17.1)$$

O operador-chave é o **atraso de uma amostra**: z^{-1} significa “o valor de uma amostra atrás”. Isso é tudo o que um microcontrolador precisa – guardar valores anteriores.

17.2 Relação entre s e z

A ponte exata é

$$z = e^{sT} \quad (17.2)$$

Ela mapeia o plano- s no plano- z assim:

semiplano esquerdo de s (estável)	→ interior do círculo unitário de z
eixo imaginário de s	→ círculo unitário ($ z = 1$)
semiplano direito de s (instável)	→ fora do círculo unitário

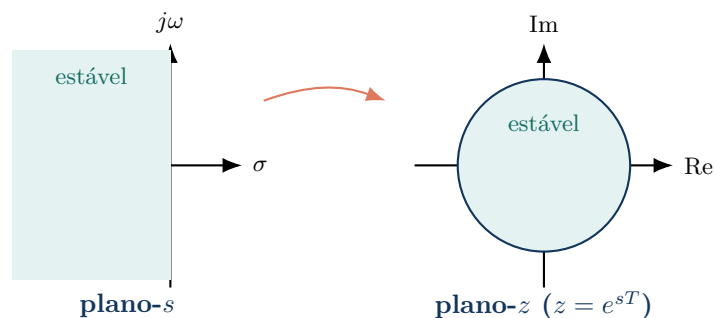


Figura 17.1: O mapa $z = e^{sT}$ “enrola” o semiplano estável de s no interior do círculo unitário de z .

Intuição

Em controle contínuo perguntamos “os polos estão à esquerda?”. Em controle digital, a pergunta vira “os polos estão **dentro do círculo unitário**?”. É o mesmo conceito de estabilidade, vestido para o tempo discreto.

CAPÍTULO 18

Discretização

Para rodar o PID contínuo no ESP32, aproximamos as operações de cálculo por fórmulas que usam apenas amostras. Há três métodos clássicos de aproximar a integral/derivada (isto é, o operador s).

Tabela 18.1: Métodos de discretização (aproximações de s).

Método	Aproximação de s	Característica
Euler <i>forward</i> (regressiva p/ frente)	$s \approx \frac{z-1}{T}$	Simples; pode instabilizar se T grande.
Euler <i>backward</i> (para trás)	$s \approx \frac{z-1}{Tz}$	Sempre estável; bom para a integral.
Tustin (bilinear/trapezoidal)	$s \approx \frac{2}{T} \frac{z-1}{z+1}$	Mais preciso; preserva melhor a fase.

Vantagens e desvantagens

Forward é a mais barata (só usa o valor atual), mas a menos robusta. **Backward** é incondicionalmente estável — por isso é segura para o *integrador*. **Tustin** é a mais fiel à dinâmica contínua, ao custo de um pouco mais de conta.

O que o seu firmware usa

O `pid.c` adota uma combinação pragmática, comum em microcontroladores:

- **Integral por Euler *forward*:** `p->integ += err*dt;` — i.e. $I[k] = I[k-1] + e[k]T$.
- **Derivada por diferença regressiva:** $\frac{e[k] - e[k-1]}{T}$, seguida de um filtro passa-baixa (Cap. 20).

Com $T = 0,01$ s e dinâmica lenta ($\omega_n \approx 2,2$), qualquer um dos métodos daria praticamente o mesmo resultado — a escolha é por simplicidade.

CAPÍTULO 19

O PID Discreto

19.1 Do contínuo ao discreto, termo a termo

Partindo de $u(t) = K_p e + K_i \int e + K_d \dot{e}$ e aplicando as aproximações do Cap. 18:

Proporcional. Direto: $P[k] = K_p e[k]$.

Integral (Euler forward).

$$I[k] = I[k-1] + e[k] T, \quad \text{termo} = K_i I[k]. \quad (19.1)$$

Derivativo (diferença regressiva).

$$d[k] = \frac{e[k] - e[k-1]}{T}, \quad \text{termo} = K_d d[k]. \quad (19.2)$$

19.2 A equação implementada

Juntando, a lei de controle discreta que roda a cada 10 ms é

$$\boxed{u[k] = K_p e[k] + K_i I[k] + K_d d_{lpf}[k]} \quad (19.3)$$

seguida de **saturação** e **anti-windup** (Cap. 21), onde d_{lpf} é a derivada *filtrada* (Cap. 20). A cada borda de “sem bola”, o estado (I e histórico) é **zerado** (`pid_reset`) para não arrastar lixo de uma tentativa à próxima.

CAPÍTULO 20

Filtro Derivativo

20.1 O problema: o derivativo amplifica ruído

A derivada *bruta* divide a diferença de amostras por $T = 0,01$ s — ou seja, **multiplica por 100**. Um pequeno tremor de ruído no sensor entre duas amostras vira um pico enorme no termo D, que “chuta” os motores.

20.2 A solução: passa-baixa de 1.^a ordem

Filtramos a derivada com um IIR (média exponencial):

$$d_{lpf}[k] = d_{lpf}[k-1] + \alpha(d[k] - d_{lpf}[k-1]), \quad \alpha = \frac{T}{\tau_d + T}. \quad (20.1)$$

Isto equivale, no contínuo, a $C_d(s) = \frac{K_d s}{1 + \tau_d s}$: derivativo em baixas frequências, ganho limitado em altas.

O alpha do firmware

Com $\tau_d = 0,03$ s e $T = 0,01$ s:

$$\alpha = \frac{0,01}{0,03 + 0,01} = 0,25.$$

O filtro atenua ruído acima de $\sim \frac{1}{2\pi\tau_d} \approx 5,3$ Hz, preservando a dinâmica da bola (sub-Hz). A 50 Hz, esse α era 0,4; ao dobrar a taxa para 100 Hz, ele caiu para 0,25 (mais suavização por amostra, mesmo τ_d).

```
1 deriv = (err - p->prev_err) / dt;          /* diferenca regressiva */
2 if (p->d_tau > 0.0f) {                      /* passa-baixa de 1a ordem */
3     float a = dt / (p->d_tau + dt);         /* alpha = T/(tau_d+T) */
4     p->d_lpf += a * (deriv - p->d_lpf);
5     deriv = p->d_lpf;
6 }
```

Listing 20.1: Derivada filtrada em `main/pid.c` (núcleo).

CAPÍTULO 21

Anti-Windup

21.1 Saturação e *windup*

A inclinação é limitada a $\pm 0,25$ rad. Se o erro é grande por muito tempo, o **integrador acumula** um valor enorme (“*windup*”) — mas a saída já está saturada e não melhora. Quando o erro finalmente inverte, o integrador inflado demora a “descarregar”, causando um **overshoot gigante**.

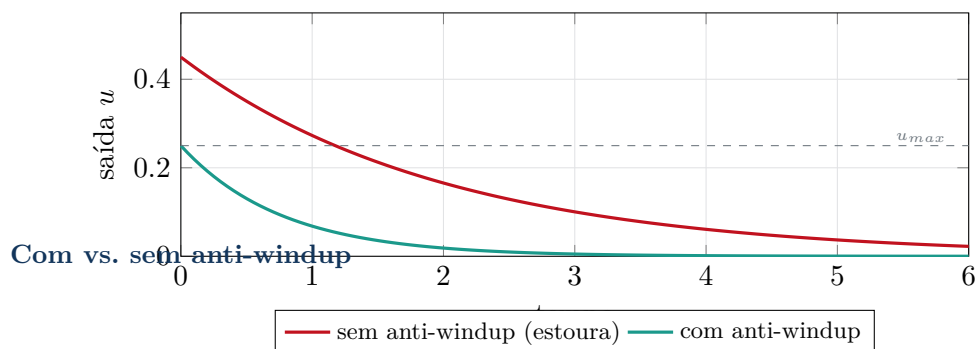


Figura 21.1: O anti-windup impede o integrador de “inflar” durante a saturação (ilustrativo).

21.2 Back-calculation

A técnica do firmware: quando a saída satura, **devolve-se ao integrador exatamente o excesso**, de modo que ele nunca acumula além do que produz a saturação:

$$\text{se } u > u_{max} : \quad I \mathrel{--} \frac{u - u_{max}}{K_i}, \quad u = u_{max}. \quad (21.1)$$

```
1 if (out > p->out_max) {
2     if (p->ki != 0.0f) p->integ -= (out - p->out_max) / p->ki;
3     out = p->out_max;
4 } else if (out < p->out_min) {
5     if (p->ki != 0.0f) p->integ -= (out - p->out_min) / p->ki;
6     out = p->out_min;
7 }
```

Listing 21.1: Anti-windup por *back-calculation* em `main/pid.c`.

Parte IV

Implementação no Projeto

CAPÍTULO 22

O Sensor: a Tela Resistiva

Sem medir a bola, não há realimentação. O sensor do projeto é uma **tela resistiva de 4 fios** (187×141 mm) sobre a qual a bola repousa.

22.1 Como funciona

A tela tem **duas camadas** condutoras separadas por micro-espaçadores. O peso da bola **encosta** uma camada na outra no ponto de contato. Para ler o eixo X, aplica-se uma tensão ao longo de uma camada (um **divisor de tensão** na direção X) e lê-se a tensão no ponto de contato pela outra camada — ela é **proporcional à posição**. Trocando o papel das camadas, lê-se Y. Dois ciclos $\Rightarrow$ coordenadas (x, y) .

Intuição

A tela é um **potenciômetro 2D**: a posição da bola vira uma fração da tensão aplicada. Barato, simples e sem câmera.

22.2 Conversão para coordenadas

O ADC do ESP32 lê a tensão como um número “cru” de 12 bits (0–4095). A calibração mapeia o cru para milímetros por uma reta:

$$x_{mm} = \frac{cru_x - cru_{min}}{cru_{max} - cru_{min}} \times 187.$$

22.3 Calibração

O firmware aprende cru_{min} e cru_{max} de cada eixo colocando a bola nos **4 cantos** (comando CAL); o **centro é derivado** deles, não medido (medi-lo “no olho” enviesaria a reta). Os valores ficam no NVS e valem após reiniciar.

22.4 Ruído e quantização

- **Ruído elétrico**: motores e o acoplamento capacitivo entre as camadas poluem a leitura. Defesas: média de 8 amostras de ADC, filtro IIR ($\alpha = 0,45$), **detecção digital** de presença (imune ao acoplamento) e **rejeição de outliers**.
- **Quantização**: 12 bits em 187 mm dão $\sim 0,05$ mm por degrau de ADC — fino o bastante; o limite prático é o ruído, não a resolução.

22.5 Impacto no controle

O sensor é o teto do desempenho

Ruído e atraso de leitura **limitam o quanto de K_d** se pode usar: como o derivativo amplifica ruído (Cap. 20), uma leitura suja obriga a filtrar mais — o que *adiciona* atraso (Cap. 16). Por isso investir na **qualidade da medida** (detecção digital, rejeição de *glitch*) foi tão decisivo quanto sintonizar o PID: melhor sensor $\Rightarrow$ mais K_d útil $\Rightarrow$ resposta mais firme.

CAPÍTULO 23

Análise do Código

Aqui a teoria encontra o C. Três arquivos formam o coração do sistema.

23.1 pid.c — o controlador por eixo

A função `pid_update` é a Eq. (19.3) em código, na ordem: derivada filtrada → integral → soma → saturação/anti-windup.

```
1 float err = meas - setpoint;          /* e[k] = medida - alvo */
2 /* derivada filtrada (Cap. 18) */
3 float deriv = 0.0f;
4 if (p->primed) {
5     deriv = (err - p->prev_err) / dt;
6     float a = dt / (p->d_tau + dt);
7     p->d_lpf += a * (deriv - p->d_lpf);
8     deriv = p->d_lpf;
9 }
10 p->prev_err = err;
11 p->integ += err * dt;                  /* integral Euler forward */
12 float out = p->kp*err + p->ki*p->integ + p->kd*deriv; /* u[k] (Eq.) */
13 /* saturacao + anti-windup (Cap. 19) ... */
```

Listing 23.1: Núcleo de `pid_update` (resumido).

Mapeamento teoria para código

$p \rightarrow kp \cdot err = K_p e[k]$; $p \rightarrow ki \cdot p \rightarrow integ = K_i I[k]$; $p \rightarrow kd \cdot deriv = K_d d_{lpf}[k]$. A estrutura `pid_t` guarda K_p, K_i, K_d , os limites de saída, o acumulador `integ`, o erro anterior `prev_err` e o estado do filtro `d_lpf`.

23.2 control.c — o laço de 100 Hz

A cada 10 ms: lê o toque (com detecção digital e rejeição de *outliers*), aplica o **debounce** de presença, roda os dois PIDs, soma o **TRIM** e converte a inclinação em ângulos.

```
1 float ox = pid_update(&s_pid_x, p.x_mm, SETPOINT_X_MM, dt_real);
2 float oy = pid_update(&s_pid_y, p.y_mm, SETPOINT_Y_MM, dt_real);
3 nx = s_sign_x * ox;          /* realimentacao negativa: TILT_SIGN = -1 */
4 ny = s_sign_y * oy;
5 nx += s_trim_x;              /* feedforward de nivel (base torta) */
6 ny += s_trim_y;
7 apply_tilt(nx, ny, thoff);    /* cinematica inversa 3RPS -> 3 motores */
```

Listing 23.2: Trecho central de `control.c` (saída do PID → motores).

23.3 kinematics.c — a cinemática inversa 3RPS

A função `machine_theta` recebe o vetor normal $(n_x, n_y, 1)$ da mesa e devolve o **ângulo de cada motor**. Ela normaliza a normal, posiciona a junta esférica de cada perna e resolve o triângulo braço-biela por lei dos cossenos (daí os dois *acos*):

```

1 double nmag = sqrt(nx*nx + ny*ny + 1.0); /* normaliza (nx,ny,1) */
2 nx /= nmag; ny /= nmag; double nz = 1.0/nmag;
3 /* ... posiciona a junta da perna (y, z) a partir da normal ... */
4 mag = sqrt(y*y + z*z);
5 angle = acos(y/mag) /* geometria */
6 + acos((mag*mag + f*f - g*g)/(2.0*mag*f)); /* lei dos cossenos */
7 return angle * (180.0/M_PI); /* radianos -> graus */

```

Listing 23.3: Esqueleto de machine_theta (perna A).

Do ângulo para passos

A saída em graus vira passos pelo fator $\frac{200 \cdot \text{microsteps}}{360^\circ}$. Com microstepping 1/16:

$$\frac{200 \cdot 16}{360} \approx 8,9 \text{ passos por grau.}$$

É o elo final entre o “mundo contínuo” do controle e os passos discretos do motor.

CAPÍTULO 24

Arquitetura Completa do Sistema

24.1 O fluxo de ponta a ponta

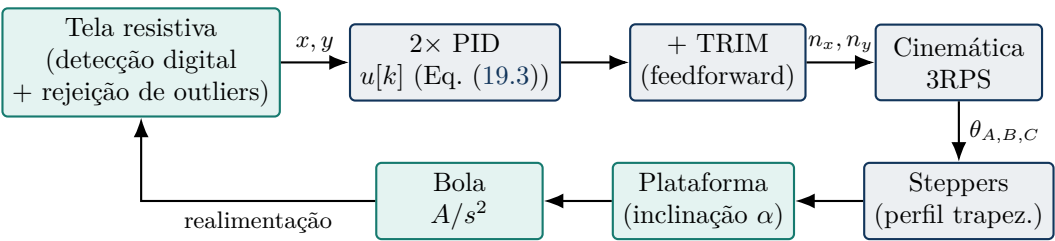


Figura 24.1: Arquitetura completa: o anel fecha do sensor à bola e volta, a 100 Hz.

24.2 Camadas de software

Tabela 24.1: Módulos do firmware e seu papel.

<code>main.c</code>	Inicialização; seleciona modo CONTROL/MAPPING.
<code>control.c</code>	Laço de 100 Hz, <i>homing</i> , parser serial, TRIM, telemetria.
<code>pid.c</code>	PID por eixo (derivada filtrada + anti-windup).
<code>kinematics.c</code>	Cinemática inversa 3RPS (<i>machine_theta</i>).
<code>steppers.c</code>	Geração de passos STEP/DIR (perfil trapezoidal, 10 kHz).
<code>touch_screen.c</code>	Leitura resistiva, detecção digital, filtro, outliers.
<code>cal_store.c</code>	Persistência em NVS (PID, calibração, curso, ZERO, TRIM).

24.3 O atuador: perfil trapezoidal

Cada motor segue a posição-alvo com um **perfil trapezoidal de velocidade**: acelera até `SPEED_BALANCE`, navega, e desacelera a tempo de parar *exatamente* no alvo (velocidade segura $v_{lim} = \sqrt{2ad}$). Sem essa rampa, a partida brusca causaria **perda de passos**. A banda do atuador (centenas de Hz) é muito maior que a da bola ($\sim 0,35$ Hz), por isso ele **não compromete** a estabilidade projetada e pode ser tratado como ganho quase-estático na análise (Cap. 7).

CAPÍTULO 25

Guia Prático de Sintonia

Todo o ajuste é feito **ao vivo pelo terminal serial** (USB-Serial-JTAG), sem recompilar. Ligue ERROR para ver e_x, e_y e a distância ao centro a cada segundo.

25.1 Roteiro recomendado

1. **Ajuste K_p :** com $K_i = K_d = 0$, suba K_p até a bola reagir bem, porém *oscilando* um pouco. (KP 0.0006...)
2. **Ajuste K_d :** adicione derivativo até a oscilação **sumir** (alvo: $\zeta \approx 1$, sem overshoot). *É o passo que estabiliza* o duplo integrador. (PID - - 0.012)
3. **Ajuste K_i :** um **toque** para eliminar o desvio residual da mesa torta. (PID - 0.0001 -)
4. **TRIM:** com a bola centrada, TRIM e TRIM SAVE (Cap. 26).
5. **Grave:** PID SAVE.

25.2 Reconhecendo cada patologia

Tabela 25.1: Diagnóstico rápido por sintoma.

Sintoma	Causa provável	Ação
Oscilação crescente	K_p alto / K_d baixo	subir K_d ou baixar K_p
Instabilidade (“surta”)	ζ muito baixo	subir K_d ; checar outliers
Lentidão	K_p baixo / K_d altíssimo	subir K_p / baixar K_d
Desvio fixo do centro	K_i fraco / base torta	subir K_i ; usar TRIM
Saturação constante	ganhos altos demais	reduzir; conferir anti-windup
Tremor /micro-movimento	ruído / K_d alto	filtrar mais; baixar K_d

25.2.1 Assinaturas visuais (o que olhar na bancada)

- **Excesso de K_p :** a bola fica **nervosa** — vibra/oscila em torno do centro com amplitude que não morre. *Baixe K_p ou suba K_d .*
- **Falta de K_d :** a bola **passa do centro** e volta várias vezes (overshoot grande) antes de assentar; a qualquer empurrão, “surta”. *Suba K_d .*
- **Excesso de K_i :** **oscilação lenta e larga** (vai longe de um lado, depois do outro), com período de vários segundos — é o *windup*. *Baixe K_i .*
- **Falta de K_i :** a bola estabiliza, porém **deslocada** do centro (mesa torta). *Suba K_i ou use TRIM.*

A lição do seu próprio rig

Quando o K_d salvo ficou cerca de $120\times$ abaixo do necessário, a mesa “surtava do nada” — $\zeta \approx 0,17$ (Cap. 11). A cura foi exatamente a teoria: **mais amortecimento** (subir K_d) e **rejeição de leituras espúrias**. Confie no ζ : ele prevê o comportamento antes de você tocar na mesa.

25.3 Teoria vs. Hardware: por que os ganhos não batem

Os ganhos calculados por alocação de polos (Cap. 10) quase nunca são os ganhos *fnais* no rig — e isso **não é um erro**. O modelo $P(s) = A/s^2$ é ideal; o hardware carrega imperfeições que o modelo ignora.

Tabela 25.2: Por que o ganho de bancada difere do ganho de projeto.

Fator real	Efeito sobre os ganhos
Ruído do sensor	limita K_d (o derivativo amplifica ruído)
Saturação ($\pm 0,25$ rad)	ganhos altos saturam mais cedo; <i>não</i> aceleram
Atrasos (amostragem, motor)	consomem margem de fase; pedem cautela com K_p
Limites mecânicos (folga, atrito, base torta)	criam erros que o modelo ideal não prevê
Cinemática não linear	o ganho efetivo varia com a inclinação

A teoria acerta o ponto de partida e nao o ponto final

A alocação de polos diz *onde começar* e *para que lado andar*: se oscila, falta K_d ; se desvia, falta K_i . O valor exato sai do ajuste fino no hardware. É como uma receita: as proporções vêm do livro, o “ponto” do sal você acerta provando.

Preparado para a banca

Se perguntarem “*por que seus ganhos não são os calculados?*”, a resposta é: o modelo é **linear, ideal e sem atraso**; o rig tem ruído, saturação, atraso e folga mecânica. A teoria deu o **projeto inicial** (estrutura do controlador, ordem de grandeza e direção de ajuste) e validou o comportamento em simulação; o conjunto final foi **refinado experimentalmente**, apoiado na rejeição de ruído e no *feedforward* TRIM. Teoria e prática não se contradizem — complementam-se.

CAPÍTULO 26

Melhorias Futuras

O PID resolve muito bem o Ball Balancer, mas a teoria moderna oferece extensões.

26.1 Feedforward (já presente: o TRIM)

A base torta é uma **perturbação constante**. Em vez de esperar o integral lento descobri-la a cada bola, o **TRIM** mede uma vez a inclinação que o integral encontrou e a aplica como *feedforward* fixo (salvo no NVS).

$$n_x \leftarrow \underbrace{s_x u_x}_{\text{PID}} + \underbrace{\text{TRIM}_x}_{\text{feedforward}} \quad (26.1)$$

Intuição

Feedforward é “saber a resposta de antemão”. Como a mesa está sempre torta do mesmo jeito, não faz sentido redescobrir isso a cada bola: aplicamos a correção direto, e o PID só cuida do que *varia*. É separar a rejeição da perturbação DC da regulação em torno dela.

26.2 Para além do PID (menção breve)

O PID resolve muito bem este sistema. Se um dia se quiser ir adiante, ficam as **citações** a seguir — *fora do escopo* desta apostila, que foca no domínio completo do PID digital atual:

- **Espaço de estados / LQR:** projeta o controlador tratando posição e velocidade como “estado”, de forma ótima e sistemática — generaliza a alocação de polos do Cap. 10.
- **Filtro de Kalman:** estima a velocidade de forma ótima a partir da medida ruidosa, em vez da derivada numérica (Cap. 20).
- **MPC:** otimiza a ação respeitando *explicitamente* a saturação de $\pm 0,25$ rad.
- **Controle adaptativo:** reajusta os ganhos sozinho conforme a planta muda — a família do PIDAUTO embarcado.

Todas **complementam**, não substituem, o PID consolidado aqui.

Tabela 26.1: Quando valeria a pena cada técnica avançada.

Técnica	Ganho potencial sobre o PID atual
Feedforward (TRIM)	já implementado: centra na hora, sem rampa do integral.
LQR / espaço de estados	Sintonia sistemática e ótima; útil se acoplar X–Y.
Kalman	Velocidade limpa $\Rightarrow$ pode subir K_d sem ruído.
MPC	Trata saturação e restrições de forma ótima.
Adaptativo	Robustez a mudanças (peso da bola, atrito).

Mensagem final

Você tem, hoje, um sistema **completo e funcional**: planta modelada, ganhos com base teórica, PID digital com filtro e anti-windup, cinemática 3RPS, rejeição de ruído e *feedforward* para a base torta. As “melhorias futuras” não são conserto — são os *próximos capítulos* naturais de quem já domina o básico. Boa defesa de projeto!

Referências Bibliográficas

- [1] G. F. Cordeiro, *Controle Clássico da Planta Ball Balancer*. Trabalho de graduação em engenharia elétrica, UNESP/FEIS, Ilha Solteira, 2022.
- [2] K. Ogata, *Engenharia de Controle Moderno*. Pearson Prentice Hall, 5 ed., 2010.
- [3] N. S. Nise, *Engenharia de Sistemas de Controle*. LTC, 7 ed., 2017.
- [4] G. F. Franklin, J. D. Powell, and M. Workman, *Digital Control of Dynamic Systems*. Addison-Wesley, 3 ed., 1998.
- [5] K. J. Åström and R. M. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press, 2008.
- [6] A. Musa, “Ball balancer.” Instructables, 2021. Disponível em: <https://www.instructables.com/Ball-Balancer/>.
- [7] Espressif Systems, *ESP32-S3 Technical Reference Manual*, 2023.

Índice Remissivo

aliasing, 29
alocação de polos, 19
amostragem, 29
anti-windup, 35
arquitetura, 41
atraso, 30

Ball Balancer, 2

controle digital, 29

diagrama de blocos, 15
discretização, 32
domínio z , 31

estabilidade, 16

feedforward, 44
filtro derivativo, 34
firmware, 39
função de transferência, 12

instabilidade, 2

Kalman, 44
 K_d , 18
 K_i , 18
 K_p , 17

Laplace, 11
linearização, 9
LQR, 44

modelagem, 6
MPC, 44

Nyquist, 29

ω_n , 22
overshoot, 24

perfil trapezoidal, 41
PID, 17
PID discreto, 33
planta, 12
polos, 12

quantização, 29

realimentação negativa, 15
revisão matemática, 4
rolamento sem escorregamento, 6

ruído, 34

saturação, 35
sensor, 37
sintonia, 42
sobressinal, 24

tela resistiva, 37
tempo de acomodação, 25
teoria vs hardware, 43
transformada Z , 31
TRIM, 44
tuning, 42

windup, 35

zeros, 12
 ζ , 22
Ziegler-Nichols, 26